El modelo anémico te está mintiendo

Domain-Driven Design en .NET y frontend

Ponentes: Jose Luis Avila · Brandon Rodriguez **Duración**: 30 minutos

---

Resumen ejecutivo

Esta charla expone los problemas de diseño más comunes en aplicaciones .NET empresariales y cómo Domain-Driven Design ofrece soluciones concretas y pragmáticas. Jose Luis Avila cubre los patrones tácticos de DDD implementados en C# con EF Core, con énfasis en los errores más frecuentes y sus correcciones. Brandon Rodriguez extiende los mismos principios al frontend.

El proyecto de demostración es **CourseCraft**, una plataforma de cursos online simplificada que sirve como hilo conductor de toda la charla, ilustrando los problemas y soluciones en un contexto reconocible para cualquier desarrollador.

Estructura de la charla

Bloque	Ponente	Tema	Duración
1	Ambos	Apertura y presentación de CourseCraft	2 min
2	Jose Luis Avila	El modelo anémico: el anti-patrón silencioso de .NET	4 min
3	Jose Luis Avila	Ubiquitous Language: el problema político antes que técnico	3 min
4	Jose Luis Avila	Bloques tácticos en C# con EF Core	8 min
5	Transición	El dominio no termina en el backend	1 min
6	Brandon Rodriguez	DDD en el frontend	5 min
7	Ambos	Bounded Contexts y Domain Events: el puente backend-frontend	5 min
8	Ambos	Cierre: las tres mentiras que se cuentan sobre DDD	2 min

---

Proyecto de demostración: CourseCraft

CourseCraft es una plataforma de cursos online (similar a Udemy) con cinco Bounded Contexts claramente delimitados:

Contexto	Responsabilidad principal
Catálogo	Creación y publicación de cursos
Inscripción	Compra y acceso de estudiantes
Aprendizaje	Progreso y reproducción de lecciones

Certificación	Emisión de certificados al completar un curso
Notificaciones	Comunicaciones a instructores y estudiantes

CourseCraft fue elegido como proyecto de demostración porque concentra los problemas que DDD resuelve: el concepto "Curso" tiene significados distintos para distintos equipos, las reglas de negocio están dispersas en múltiples clases de servicio, y EF Core usado sin disciplina de dominio termina dictando cómo se diseña el modelo.

Guión de la charla

Bloque 1 — Apertura y proyecto demo (2 min)

La charla abre mostrando un fragmento de código que la mayoría de asistentes reconocerá de inmediato:

```
public class UserService
{
    public void CreateUser(string name, string email, string roleId) { ... }
    public void UpdateUser(int id, string name, string email) { ... }
    public void DeactivateUser(int id) { ... }
    public void AssignRole(int userId, string roleId) { ... }
    public void SendVerificationEmail(int userId) { ... }
    public void ResetPassword(int userId, string newPassword) { ... }
    public bool ValidateCredentials(string email, string password) { ... }
    // ... 20 métodos más
}
```

Este patrón — una clase de servicio que acumula toda la lógica del sistema — es el síntoma del problema central que se desarrollará en la charla. Se establece como punto de partida antes de presentar CourseCraft como el contexto donde se demostrarán las soluciones.

> "*Voy a apostar que la mayoría de los proyectos .NET en los que han trabajado tienen una clase que se llama algo así. Este es el síntoma. En 30 minutos van a entender exactamente por qué pasa y cómo salir de ahí.*"

Bloque 2 — El modelo anémico (4 min)

El problema

El modelo anémico es el anti-patrón más extendido en proyectos .NET, y paradójicamente es el que promueven la mayoría de tutoriales de EF Core Code First. Las entidades solo contienen propiedades; toda la lógica de negocio reside en clases de servicio separadas.

```
// Entidad anémica — solo datos, sin comportamiento
public class Curso
{
    public int Id { get; set; }
    public string Titulo { get; set; }
    public decimal Precio { get; set; }
    public bool EstaPublicado { get; set; }
    public List<Leccion> Lecciones { get; set; }
    public int InstructorId { get; set; }
}
```

```
// Toda la lógica vive aquí, separada de los datos que protege
public class CursoService
{
    public void PublicarCurso(int cursoId)
    {
        var curso = _repo.GetById(cursoId);

        if (curso.Lecciones.Count < 5)
            throw new Exception("Mínimo 5 lecciones");
        if (curso.Precio <= 0)
            throw new Exception("Precio inválido");

        curso.EstaPublicado = true;  // Backdoor directo, sin validación
        _repo.Save(curso);
    }
}
```

El riesgo concreto: `curso.EstaPublicado = true` desde cualquier Controller o servicio salta todas las validaciones. EF Core lo permite sin advertir.

La solución: dominio rico

```
public class Curso
{
    private readonly List<Leccion> _lecciones = new();
    public IReadOnlyCollection<Leccion> Lecciones => _lecciones.AsReadOnly();

    public string Titulo { get; private set; }
    public Dinero Precio { get; private set; }
    public EstadoCurso Estado { get; private set; }

    public void Publicar()
    {
        if (_lecciones.Count < 5)
            throw new DomainException("Un curso debe tener al menos 5 lecciones para
publicarse");

        if (Estado == EstadoCurso.Publicado)
            throw new DomainException("El curso ya está publicado");

        Estado = EstadoCurso.Publicado;
    }

    public void AgregarLeccion(string titulo, TimeSpan duracion)
    {
        if (Estado == EstadoCurso.Publicado)
            throw new DomainException("No se pueden agregar lecciones a un curso
publicado");

        _lecciones.Add(new Leccion(titulo, duracion));
```

```
    }
}
```

Los cambios respecto al modelo anémico:

- Los setters son privados: `curso.EstaPublicado = true` ya no compila desde afuera
- La colección de lecciones es de solo lectura: no se puede modificar sin pasar por el aggregate
- Las reglas de negocio viven junto a los datos que protegen
- El método `Publicar()` documenta con exactitud qué significa publicar un curso

> *"El modelo anémico no es un problema de código malo. Es un problema de que EF Core hace que sea más fácil hacerlo mal que hacerlo bien."*

Bloque 3 — Ubiquitous Language (3 min)

El lenguaje ubicuo no es un ejercicio de naming. Es el resultado de conversaciones con el negocio que revelan ambigüedades fundamentales en cómo se entiende el dominio.

El problema en CourseCraft

¿Qué es un "estudiante" en este sistema?

- El equipo de **ventas** responde: cualquier persona que compró algún curso
- El equipo de **soporte** responde: cualquier persona con cuenta activa
- El equipo de **aprendizaje** responde: alguien activo en un curso específico

Las tres respuestas son válidas. El problema es que el código no lo refleja, y termina acumulando ambigüedad:

```csharp
public class UserService
{
    public List<User> GetStudents() { }              // ¿Cuáles?
    public List<User> GetActiveStudents() { }        // ¿Activos en qué?
    public List<Enrollment> GetStudentCourses() { }  // ¿Ahora es Enrollment?
}
```

La solución: términos distintos por Bounded Context

```csharp
// En el contexto de Ventas
public class Comprador { }       // quien realizó una transacción de pago

// En el contexto de Aprendizaje
public class Inscripcion { }     // relación activa entre usuario y curso específico

// En el contexto de Soporte
public class UsuarioCuenta { }   // tiene credenciales activas, puede necesitar ayuda
```

El criterio de code review que se propone: si el nombre de una clase requiere una explicación oral para entenderse, el lenguaje ubicuo ha fallado.

Bloque 4 — Bloques tácticos en C# con EF Core (8 min)

4.1 Value Objects

El error más frecuente al aprender DDD: crear Value Objects que son solo wrappers de primitivos sin validación ni comportamiento.

```
// Sin valor agregado real
public record CursoId(Guid Value);
public record UsuarioId(Guid Value);
public record LeccionId(Guid Value);
```

Un Value Object justifica su existencia cuando encapsula validación o comportamiento, eliminando la necesidad de repetir esa lógica en múltiples lugares del código:

```
public record Email
{
    public string Valor { get; }

    public Email(string valor)
    {
        if (string.IsNullOrWhiteSpace(valor))
            throw new DomainException("El email no puede estar vacío");
        if (!valor.Contains('@'))
            throw new DomainException($"'{valor}' no es un email válido");

        Valor = valor.Trim().ToLowerInvariant();
    }
}

public record Dinero
{
    public decimal Monto { get; }
    public string Moneda { get; }

    public Dinero(decimal monto, string moneda)
    {
        if (monto < 0) throw new DomainException("El monto no puede ser negativo");
        Monto = monto;
        Moneda = moneda.ToUpperInvariant();
    }

    public Dinero Aplicar(Descuento descuento) =>
        new(Monto * (1 - descuento.Porcentaje), Moneda);

    public Dinero Sumar(Dinero otro)
    {
        if (Moneda != otro.Moneda)
            throw new DomainException("No se pueden sumar monedas distintas");
        return new(Monto + otro.Monto, Moneda);
    }
}
```

Configuración en EF Core con Owned Types:

```
modelBuilder.Entity<Curso>()
    .OwnsOne(c => c.Precio, precio =>
    {
        precio.Property(p => p.Monto).HasColumnName("precio_monto");
        precio.Property(p => p.Moneda).HasColumnName("precio_moneda");
    });
// EF aplana el Value Object en la misma tabla del Curso — sin tabla extra, sin FK
```

4.2 Aggregates y la decisión del tamaño

Un Aggregate define cuántos objetos se cargan y guardan en la misma transacción. La pregunta que guía la decisión:

> *¿Existe una regla de negocio que requiere consistencia entre estos dos objetos en la misma operación?*

- Si la respuesta es **sí**: van en el mismo Aggregate
- Si la respuesta es **no**: son Aggregates distintos, se referencian por ID

```
// Leccion como Aggregate independiente
public class Leccion
{
    public LeccionId Id { get; }
    public CursoId CursoId { get; }   // Referencia por ID — no carga el Curso completo
    public string Titulo { get; private set; }

    public void Renombrar(string nuevoTitulo)
    {
        if (string.IsNullOrWhiteSpace(nuevoTitulo))
            throw new DomainException("El título no puede estar vacío");
        Titulo = nuevoTitulo;
    }
}
```

Cuándo `Leccion` va dentro de `Curso`: cuando existe la regla "un curso publicado debe tener al menos 5 lecciones". Esa validación requiere que ambos estén en el mismo Aggregate.

Cuándo `Leccion` es un Aggregate independiente: cuando "un estudiante marca una lección como completada". Esa operación no necesita cargar ni validar el curso completo.

4.3 EF Core y DDD: configuración avanzada

```
modelBuilder.Entity<Curso>(builder =>
{
    // Backing fields para colecciones privadas
    builder.Navigation(c => c.Lecciones)
           .UsePropertyAccessMode(PropertyAccessMode.Field);

    // Estado persistido como string — legible en la BD sin joins
    builder.Property(c => c.Estado)
           .HasConversion<string>();

    // Value converter para tipos de dominio custom
```

```
    builder.Property(c => c.InstructorId)
            .HasConversion(
                id => id.Value,
                value => new InstructorId(value)
            );
});
```

El constructor privado para hidratación por EF Core:

```
public class Curso
{
    // Constructor de dominio — con todas las validaciones del negocio
    public Curso(string titulo, InstructorId instructor, Dinero precio)
    {
        if (string.IsNullOrWhiteSpace(titulo))
            throw new DomainException("El título es requerido");
        // ...
    }

    // Constructor privado para EF Core — sin validaciones
    // EF reconstruye el objeto desde BD; los datos ya estaban validados al persistirse
    private Curso() { }
}
```

4.4 MediatR como arquitectura de casos de uso

```
// Command — representa la intención del usuario, no un DTO de request HTTP
public record PublicarCursoCommand(Guid CursoId, Guid InstructorId) : IRequest<Result>;

// Handler — orquesta el caso de uso sin tener lógica de negocio propia
public class PublicarCursoHandler : IRequestHandler<PublicarCursoCommand, Result>
{
    public async Task<Result> Handle(PublicarCursoCommand request, CancellationToken ct)
    {
        var curso = await _cursoRepo.GetByIdAsync(new CursoId(request.CursoId));

        if (curso is null) return Result.Failure("Curso no encontrado");
        if (curso.InstructorId.Value != request.InstructorId)
            return Result.Failure("No autorizado");

        curso.Publicar();  // El dominio ejecuta la lógica de negocio

        await _cursoRepo.SaveAsync(curso);

        return Result.Success();
    }
}
```

La separación de responsabilidades que MediatR facilita:

- El **Handler** orquesta: carga, verifica autorización, delega al dominio, guarda
- El **Aggregate** ejecuta: toda la lógica de negocio, sin conocer HTTP, EF Core ni MediatR

Bloque 5 — Transición (1 min)

El backend puede tener el dominio bien modelado. Pero eso no responde si el frontend tiene dominio propio o si es solo una pantalla que muestra lo que dice el backend.

Brandon Rodriguez aborda esa pregunta desde el lado del cliente.

Bloque 6 — DDD en el frontend (5 min)

Brandon Rodriguez expone cómo los mismos principios de DDD se aplican en el frontend. Los temas incluyen la frontera API como Anti-Corruption Layer implícita, Value Objects en TypeScript con la misma filosofía que en .NET, y hooks como equivalentes de los Application Services del backend. Se desarrolla también la importancia de que el lenguaje ubicuo cruce la frontera: si el backend habla de `Inscripcion`, el frontend no debería llamarlo `enrollment`.

Bloque 7 — Bounded Contexts y Domain Events (5 min)

Los Domain Events no son mensajería interna del backend. Son el protocolo de comunicación de toda la aplicación, incluyendo el cliente.

```
// Evento definido en el dominio
public record CursoPublicado(
    CursoId CursoId,
    string Titulo,
    InstructorId InstructorId,
    DateTimeOffset OcurridoEn
) : IDomainEvent;

// El contexto de Notificaciones reacciona — sin que Catálogo lo sepa
public class NotificarInstructorHandler : INotificationHandler<CursoPublicado>
{
    public async Task Handle(CursoPublicado ev, CancellationToken ct)
        => await _email.EnviarNotificacion(ev.InstructorId, ev.Titulo);
}

// El contexto de Búsqueda también reacciona — también sin que Catálogo lo sepa
public class IndexarCursoHandler : INotificationHandler<CursoPublicado>
{
    public async Task Handle(CursoPublicado ev, CancellationToken ct)
        => await _searchIndex.IndexarAsync(ev.CursoId);
}
```

Los Bounded Contexts se comunican a través de eventos, no de llamadas directas. Catálogo no conoce a Notificaciones ni a Búsqueda. Cada contexto reacciona de forma independiente al mismo evento.

Cuando los eventos llegan al frontend vía SignalR, el cliente reacciona en tiempo real sin necesidad de polling:

Evento	Reacción en el frontend

`CursoPublicado`	Actualiza el catálogo en tiempo real
`MatriculaCreada`	Habilita el acceso al contenido
`LeccionCompletada`	Actualiza la barra de progreso

Bloque 8 — Las tres mentiras de DDD (2 min)

Mentira 1: "DDD es solo para sistemas grandes"

Los patrones tácticos — Ubiquitous Language y Value Objects en particular — aportan valor a cualquier escala desde el primer día de un proyecto. Lo que crece con el tamaño del sistema son los Bounded Contexts y el diseño estratégico. No es necesario implementar todos los patrones para obtener beneficios concretos.

Mentira 2: "El dominio no debe conocer nada de infraestructura"

En proyectos reales existe pragmatismo. Un atributo de EF Core en una entidad de dominio no arruina la arquitectura. Lo que sí la arruina es lógica de negocio en un DbContext, en un Controller, o en una migración SQL. La distinción real: el dominio debe ignorar *cómo* se persiste. No necesariamente ignorar *que* se persiste.

Mentira 3: "Si haces DDD bien, la arquitectura emerge sola"

DDD provee herramientas para diseñar bien, pero las decisiones difíciles siguen siendo responsabilidad del equipo: cuándo una validación es lógica de dominio versus lógica de aplicación, cuándo un Bounded Context se convierte en microservicio, cuándo un Value Object es solo burocracia.

Cierre

> "*DDD no es sobre patrones. Es sobre tener conversaciones más honestas con el negocio, y tener el coraje de que esas conversaciones se reflejen exactamente en el código.*"

Jose Luis Avila · Brandon Rodriguez